

Git & GitHub 로컬

=== 깃 & 깃허브 시작하기 ===

1. Git & GitHub 를 사용하는 이유

개발 과정에서 코드를 기능 추가 또는 버그 수정 시점별로 기록하고, 필요할 때 언제든지 **특정 시점으로 되돌려야(롤백)** 하는 경우가 발생합니다.

- **Git:** 코드를 버전별로 기록하며 관리할 수 있는 **분산 버전 관리 시스템 (DVCS)** 툴입니다.
- **GitHub:** 이 Git 저장소를 **온라인**에 저장하여 팀원들과 코드를 공유하고 협업하기 위해 사용되는 호스팅 서비스입니다.

2. Git 환경 설정 및 기본 워크플로우

2.0. Git 설치, Github 계정 만들기

공식 사이트를 통해 Git 을 설치합니다. <https://git-scm.com/>

운영체제	설치 명령어 (예시)
Windows/macOS	공식 설치 파일 다운로드 및 실행
Ubuntu/Linux	\$ sudo apt update 후 \$ sudo apt install git

공식 사이트 github 에서 계정도 같이 생성하세요. <https://github.com/>

2.1. Git 사용자 정보 설정(이름 이메일)

git config 명령어를 통해 팀원 각자가 자신의 PC 터미널에서 다음 두 명령어를 실행하여 전역(Global) 설정을 해야 합니다.

--global: 이 PC 의 **모든 Git 프로젝트**에 이 이름과 이메일 주소를 적용하겠다는 의미입니다. (일반적으로 이 옵션을 사용합니다.)

옵션 생략: 특정 프로젝트(저장소)에만 다른 이름/이메일을 사용하고 싶을 때는 --global 옵션을 생략하고 해당 저장소 폴더에서 명령어를 실행할 수 있습니다.

```
# 사용자 이름 설정 (커밋 로그에 표시될 이름)
$ git config --global user.name "본인의 GitHub 이름 또는 실명"

# 이메일 주소 설정 (커밋과 GitHub 계정 연결에 사용) → 이걸 github 사용 시에 추가해도 됨
$ git config --global user.email "본인의 GitHub 등록 이메일 주소"

#설정 잘 되었는지 확인

$ git config --global --list
```

2.2. 로컬 저장소 초기화 (git init)

협업할 프로젝트 폴더(디렉토리)로 이동하여 터미널에서 다음 명령어를 실행합니다.

```
$ git init
```

- **결과:** 해당 디렉토리에 **.git**이라는 숨김 폴더가 생성됩니다. 이 폴더가 해당 디렉토리 내의 모든 파일을 **추적**할 수 있도록 해줍니다.(폴더가 안보인다면 폴더 설정에서 숨김파일 보이기를 해줍니다)

3. 일일 작업의 핵심: Add & Commit

3.1. 변경 사항 스테이징 (git add)

git add 명령어는 변경된 파일들을 ****Staging Area(스테이징 영역)****에 올려 **다음 커밋에 포함될 준비**를 시키는 역할을 합니다.

역할:

1. 새로 생성된 파일(Untracked files)을 Git 의 **추적 대상**으로 만듭니다.
2. 추적 중인 파일의 수정 사항을 **Staging Area**에 올려 커밋 대기 상태로 만듭니다.

명령어	설명
-----	----

\$ git add <파일명>	특정 파일만 스테이징
\$ git add <파일명 1> <파일명 2>	여러 파일 스테이징
\$ git add <폴더명>	폴더 내 모든 파일 을 재귀적으로 스테이징
\$ git add .	현재 작업 폴더 내 모든 변경 사항을 스테이징 (가장 흔하게 사용)

3.2. 현재 상태 확인 (git status)

git status 는 Git 저장소의 가장 최근 커밋과 비교하여 현재 작업 상태(로컬 파일, 스테이징 영역)를 보여줍니다.

```
$ git status
# 또는 간결하게 볼 때
$ git status -s
```

출력 상태	설명
Changes to be committed (녹색)	스테이징이 완료되어 다음 커밋에 포함될 파일 목록
Changes not staged... (빨간색/주황색)	수정되었지만 아직 git add 를 하지 않은 파일 목록
Untracked files (빨간색/주황색)	Git 이 한 번도 추적하지 않은 새로운 파일 목록

3.3. 추적 제외 설정 (.gitignore)

민감한 정보, 빌드 결과물, IDE 설정 파일 등 Git 이 추적하지 않아야 할 파일들을 지정합니다.

1. **.gitignore 파일 생성:** 작업 폴더의 최상위 디렉토리에 파일을 생성합니다.

```
$ touch .gitignore
# 편집기로 열기 (예: VS Code)
$ code .gitignore
```

2. **무시 패턴 지정:** 파일 내부에 무시할 대상(파일 또는 폴더)을 지정합니다.

```
# 예시:
/node_modules/ # 폴더 무시
```

```
*.log      # 확장자가 .log 인 모든 파일 무시
config.local.js # 특정 파일 무시
```

중요: 이미 Git 에 의해 추적되고 있는 파일은 .gitignore 에 추가해도 무시되지 않습니다.
이 경우, **git rm --cached <파일명>** 명령어를 사용하여 Git 캐시에서 먼저 제거해야 합니다.

3.4. 변경 사항 기록 및 확정 (git commit)

Staging Area 에 있는 변경 사항들을 Git 저장소의 영구적인 기록으로 확정하고 저장합니다. 커밋 시에는 반드시 어떤 변경 사항이 있었는지 메시지를 남겨야 합니다.

```
# 짧은 메시지를 바로 남기는 방법 (권장)
$ git commit -m "Feat: 새로운 로그인 기능 추가"
```

```
# 편집기를 열어 장문의 메시지를 작성하는 방법
$ git commit
```

유용한 옵션:

- `$ git commit -a -m "메시지"`: 추적 중이던 모든 파일을 자동으로 스테이징(`git add`)하고 커밋합니다. (새로운 파일 제외)
- `$ git commit --amend`: 가장 최근 커밋을 덮어씁니다. 직전 커밋 메시지를 수정하거나 누락된 파일을 추가할 때 사용됩니다.

주의: `--amend` 는 히스토리를 재작성합니다. **원격 저장소에 Push 한 커밋은** `--amend` 로 수정하는 것을 **절대 지양**해야 합니다.

4. 히스토리 확인 및 롤백 (Reset & Revert)

커밋 히스토리는 `git log` 명령어를 통해 확인하며, 이때 나오는 **커밋 해시값**은 롤백의 기준점이 됩니다.

4.1. 커밋 히스토리 확인 (git log)

```
#기본
$ git log
```

```
# 가장 유용하고 추천되는 조합 (간결, 그래프, 포인터 표시, )
$ git log --oneline --graph --decorate -n <숫자>
```

4.2. 롤백 명령어의 종류 및 차이

Git 에서 롤백(되돌리기)은 히스토리 기록 유지 여부에 따라 두 가지로 나뉩니다.

명령어	목표	히스토리 기록	사용 시점
git reset	시간을 되돌려 그 이후의 기록을 삭제합니다.	삭제/재작성	Push 전 로컬 작업 시 (협업 X)
git revert	특정 커밋을 취소하는 새로운 기록을 추가합니다.	유지 (취소 기록이 남음)	Push 후 또는 협업 중 (가장 안전)

4.3. 롤백 명령어 구문

1) 기록 삭제 방식 (git reset)

지정한 커밋 해시 **시점**으로 돌아가며, 그 이후의 모든 커밋이 히스토리에서 사라집니다.

```
# [돌아가고 싶은 시점]의 커밋 해시를 지정합니다.
$ git reset --hard <돌아가고 싶은 시점의 커밋 해시>
```

2) 기록 유지 방식 (git revert)

지정한 커밋에서 발생한 변경 사항을 **취소**하는 새로운 커밋을 만듭니다.

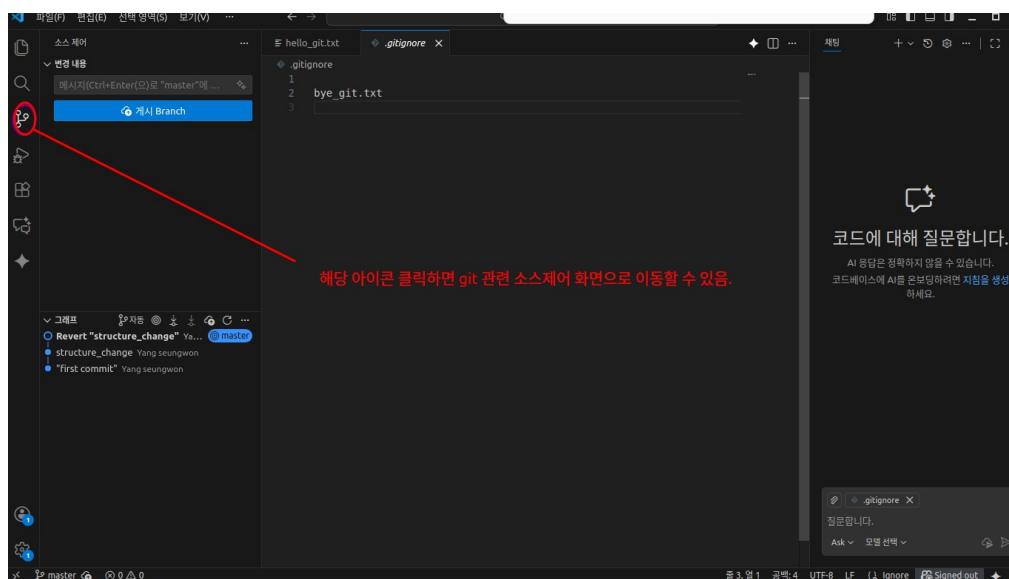
```
# [취소하고 싶은 작업]의 커밋 해시를 지정합니다.
$ git revert <취소하고 싶은 커밋 해시>
```

```
# 여러 개의 연속된 커밋을 취소하고 싶을 때 (C2 커밋 이후부터 C4 까지 취소)
# C2..C4 구문은 C2 를 제외하고 C3, C4 만 취소합니다.
$ git revert <C2_hash>..<C4_hash>
```

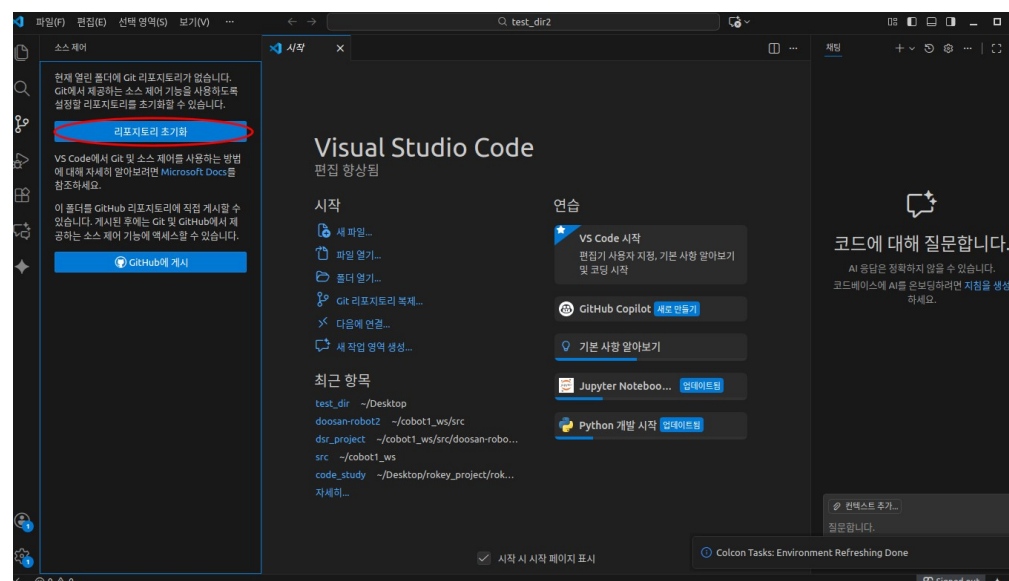
팁: 다수의 커밋을 취소하고 그 결과를 **하나의 커밋**으로 합쳐 깔끔하게 남기고 싶다면,
git revert -n <범위> 후 git commit 을 사용합니다.

4. VS Code 로 더 쉽게 이용하자 !

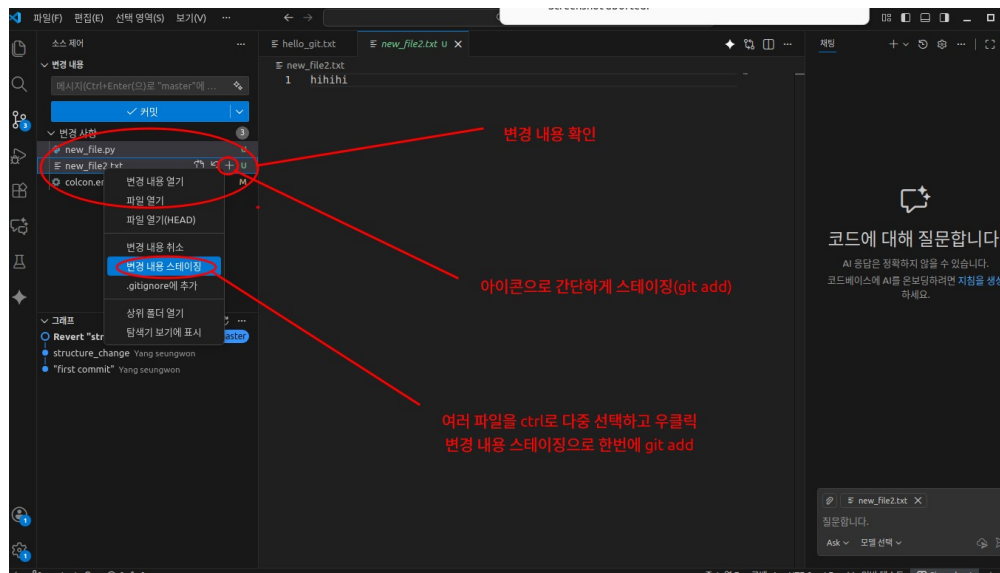
VS Code 이용하면 위 명령어들 안쓰고 쉽게 이용할 수 있습니다. 기본적으로 git 만 깔면 vscode 에 추가 확장 없이도 git 의 기능은 이용할 수 있습니다 .



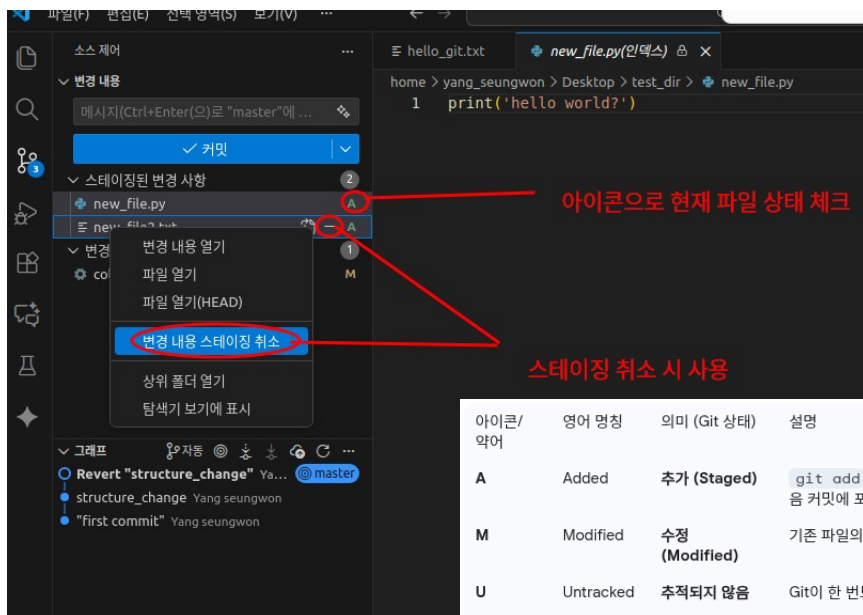
작업 폴더가 git 에 의해 추적되려면 git init 명령어를 통해 .git 폴더를 생성하여야 하는데, vscode 에서 소스제어 아이콘(위 스크린샷 아이콘)을 클릭하면 자동으로 아래와 같이 리포지토리 초기화를 선택할 수 있고, 이를 통해 .git 폴더를 생성할 수 있습니다.



만약 새로운 변경 사항이 생기면 소스제어에서 아래와 같이 변경 사항들이 확인 및 staging(git add) 가능합니다.

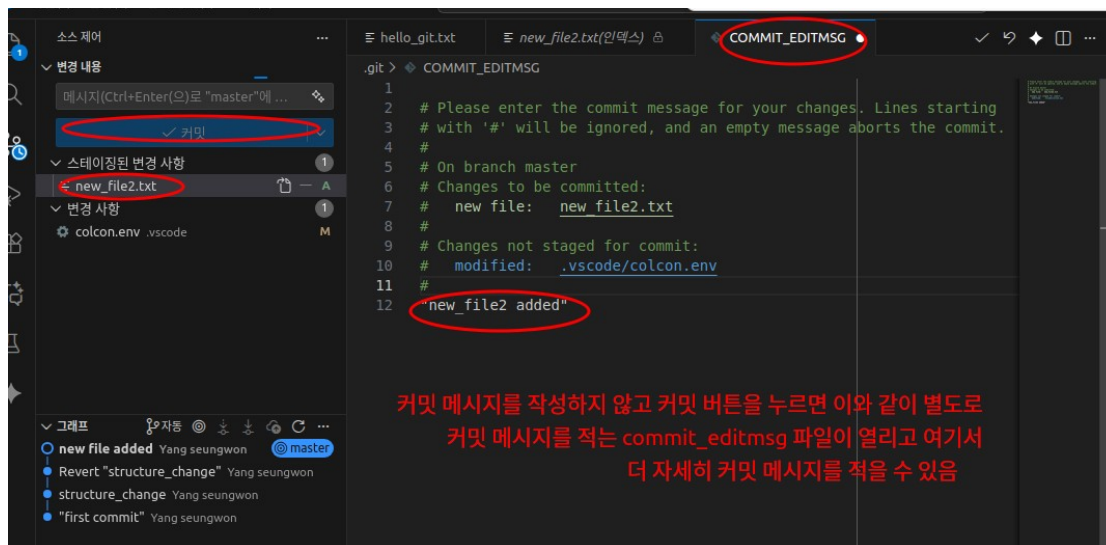
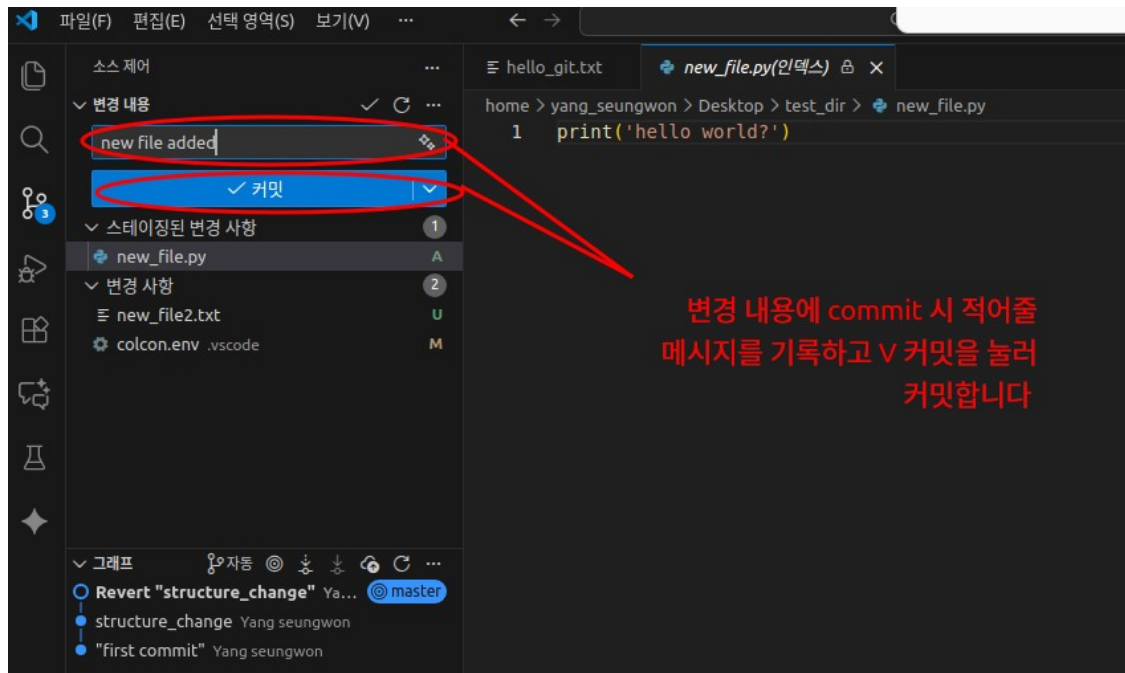


스테이징 취소는 아래와 같이 할 수 있으며, 파일 상태 아이콘은 표를 참조하세요.



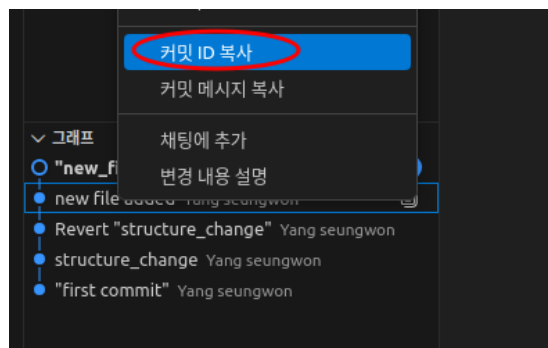
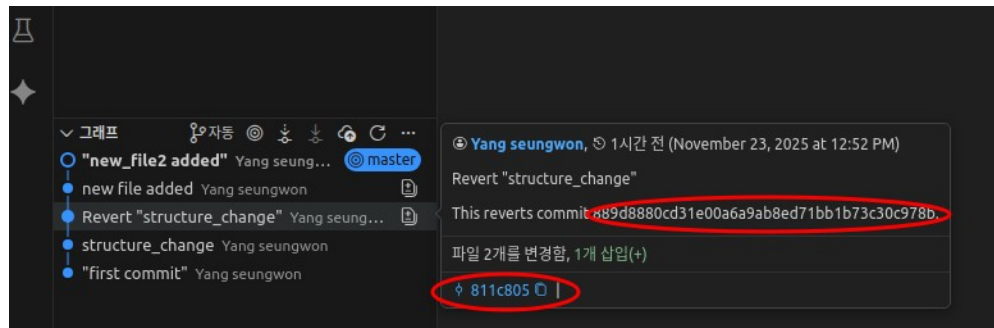
아이콘/약어	영어 명칭	의미 (Git 상태)	설명
A	Added	추가 (Staged)	git add 명령을 사용하여 새로 스테이징 영역에 추가된 파일. (다음 커밋에 포함될 예정)
M	Modified	수정 (Modified)	기존 파일의 내용이 수정되었음을 나타냅니다.
U	Untracked	추적되지 않음	Git이 한 번도 보지 못한 새로운 파일. git add 가 필요함.
D	Deleted	삭제 (Deleted)	git rm 또는 파일 삭제 후 스테이징하여 다음 커밋에서 삭제될 예정인 파일.
R	Renamed	이름 변경	파일 이름이 변경되었음을 나타냅니다.

commit 은 아래와 같은 두 방식으로 할 수 있습니다.

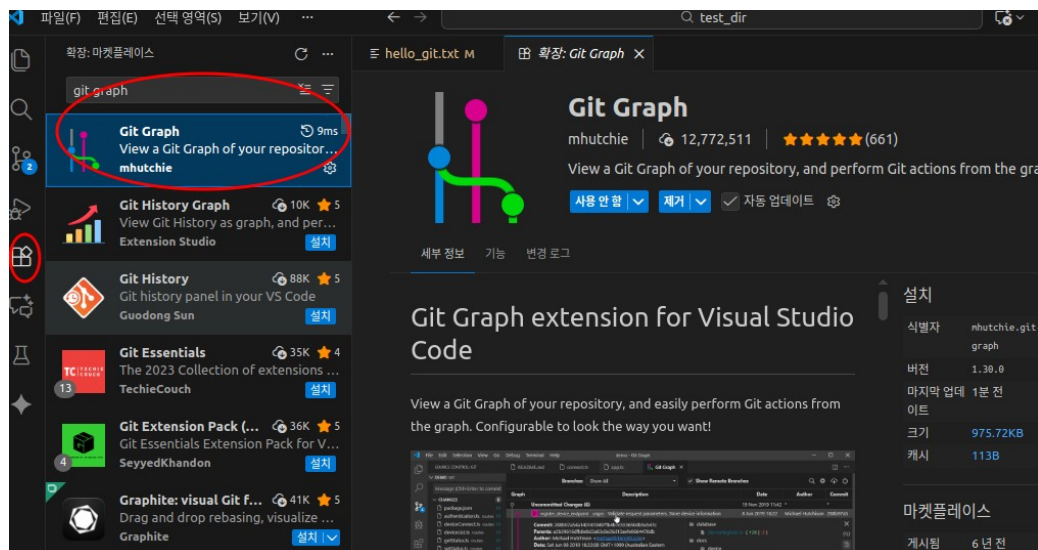


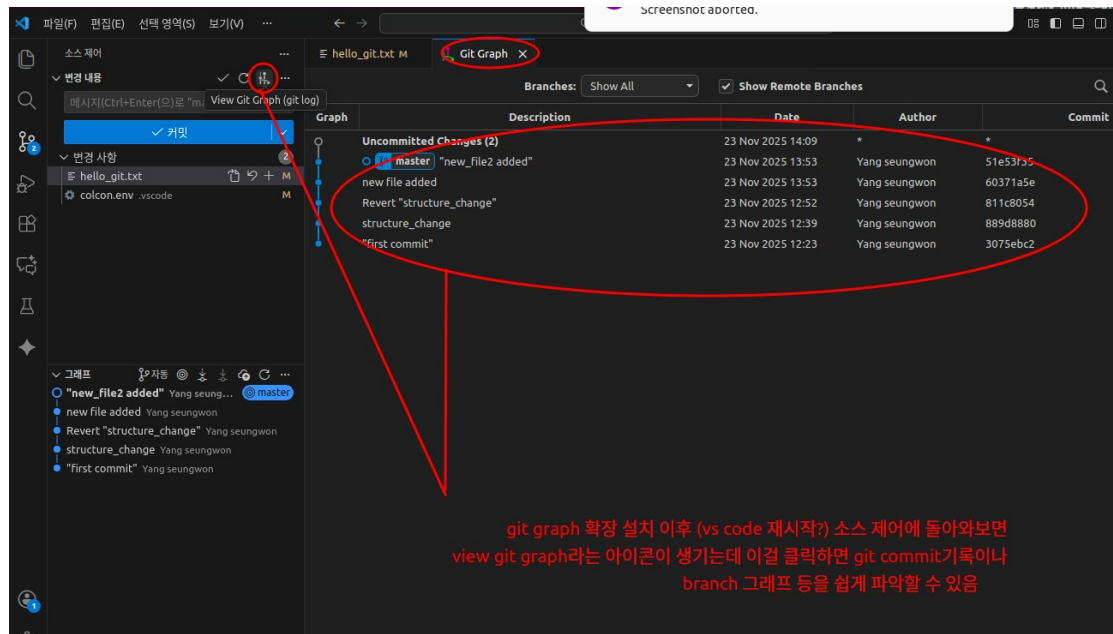
위와 같이 세부 커밋 메시지 작성 후에 저장하고 commit_editmsg 를 닫으면 알아서 커밋이 되는 방식입니다.

commit 을 하였다면 이 기록은 소스제어의 graph 탭에서 확인 할 수 있는데, 아래와 같이 단순히 커서를 올리고 커밋 해시값을 확인하던가, 우클릭하고 커밋 ID 복사로 해당 커밋 해시값을 복사할 수 있습니다.



commit 기록들과 추후 배울 branch 를 더욱 효과적으로 확인하기 위해서 'git graph'라는 확장 설치 권장됩니다.





Git graph에서 특정 commit을 우클릭해보면 아래와 같이 revert나 reset 등 롤백 관련 적용도 손쉽게 할 수 있습니다. revert의 revert(취소하면 안되는데 취소한 것 롤백)도 손쉽게 할 수 있습니다.

